Sang Ah Lee
DATA-260 Section 22
Student ID: 012634544

Configuration:
SID4: 4544
PORT_BASE: 8044
PREFIX: s4544
SEED: 4544
VERIFY_SEED: 264544
DOMAIN_ID: 4 (Open_source package vulnerabilities)

Hardware: Apple M3, 16GB
Model: qwen2.5:3b (switched from qwen3.8b due to hardware issues)

Recent repo state (w/o report): 9cf81aa5689d46e8605e369c704037c145e1ddea
Link: https://github.com/sangahlee0/data260-4544
Link to HW8 Commit#:
https://github.com/sangahlee0/data260-4544/tree/9cf81aa5689d46e8605e369c704037c145e1ddea (This commit link does not include the report, because it will be added after downloading this document)

Homework 8

PART 1- Backend Service Request/Response over Kafka

I. Operations

consumer.py



```
60-4544 > code > worker > consumer.py > process_vulnerability_request

def validate_vulnerability(data):
    # Return a readable validation error, or None when data is valid
    if not isinstance(data, dict):
        return "data must be an object"

    if not isinstance(data.get("package_id"), int) or data["package_id"] <= 0:
        return "package_id must be a positive integer"

    for field in ("vulnerability_name", "vulnerability_code"):
        if not isinstance(data.get(field), str) or not data[field].strip():
            return f"{field} is required"

    urgency_score = data.get("urgency_score")
    if not isinstance(urgency_score, int) or urgency_score < 1:
        return "urgency_score must be a positive integer"

    return None


def process_vulnerability_request(operation, data):
    # Process CREATE_VULNERABILITY or GET_VULNERABILITY
    if operation == "CREATE_VULNERABILITY":
        error = validate_vulnerability(data)
        if error:
            return {"success": False, "error": error}

        entity_id = str(uuid.uuid4())
        entity = {
            "entity_id": entity_id,
            "package_id": data["package_id"],
            "vulnerability_name": data["vulnerability_name"].strip(),
            "vulnerability_code": data["vulnerability_code"].strip(),
            "urgency_score": data["urgency_score"],
        }
        vulnerabilities[entity_id] = entity

        return {
            "success": True,
            "entity_id": entity_id,
            "message": "Entity created",
        }

    if operation == "GET_VULNERABILITY":
        if not isinstance(data, dict) or not data.get("entity_id"):
            return {"success": False, "error": "entity_id is required"}

        entity = vulnerabilities.get(data["entity_id"])
        if entity is None:
            return {"success": False, "error": "Entity not found"}

        return {"success": True, "entity": entity}

    return {"success": False, "error": "Unknown operation"}
```

II. Kafka Requirements

consumer.py

```python
async def main():
    consumer = AIOKafkaConsumer(
        REQUEST_TOPIC,
        bootstrap_servers=KAFKA_BOOTSTRAP_SERVERS,
        group_id="s4544-vulnerability-worker-group",
        auto_offset_reset="earliest",
        enable_auto_commit=True,
    )
    producer = AIOKafkaProducer(bootstrap_servers=KAFKA_BOOTSTRAP_SERVERS)

    await consumer.start()
    await producer.start()
    print("The consumer is ready and listening!")
    print(f"Listening for requests on topic: {REQUEST_TOPIC}")

    try:
        async for message in consumer:
            request = json.loads(message.value.decode())
            correlation_id = request.get("correlation_id")
            reply_to = request.get("reply_to")

            print("[WORKER] Received request from Kafka")
            print(f"[WORKER] correlation_id={correlation_id}")
            print(f"[WORKER] request={json.dumps(request, indent=2)}")

            result = process_vulnerability_request(
                request.get("operation"), request.get("data")
            )
            response = {"correlation_id": correlation_id, **result}

            print("[WORKER] Sending response to Kafka")
            print(f"[WORKER] response={json.dumps(response, indent=2)}")
            await producer.send_and_wait(reply_to, json.dumps(response).encode())
    finally:
        await consumer.stop()
        await producer.stop()


if __name__ == "__main__":
    asyncio.run(main())
```

kafka_rpc.py

```python
import asyncio
import json
import uuid

from aiokafka import AIOKafkaConsumer, AIOKafkaProducer


KAFKA_BOOTSTRAP_SERVERS = "localhost:9092"
REQUEST_TOPIC = "s4544_vulnerability_requests"
RESPONSE_TOPIC = "s4544_vulnerability_responses"
TIMEOUT_SECONDS = 8
```



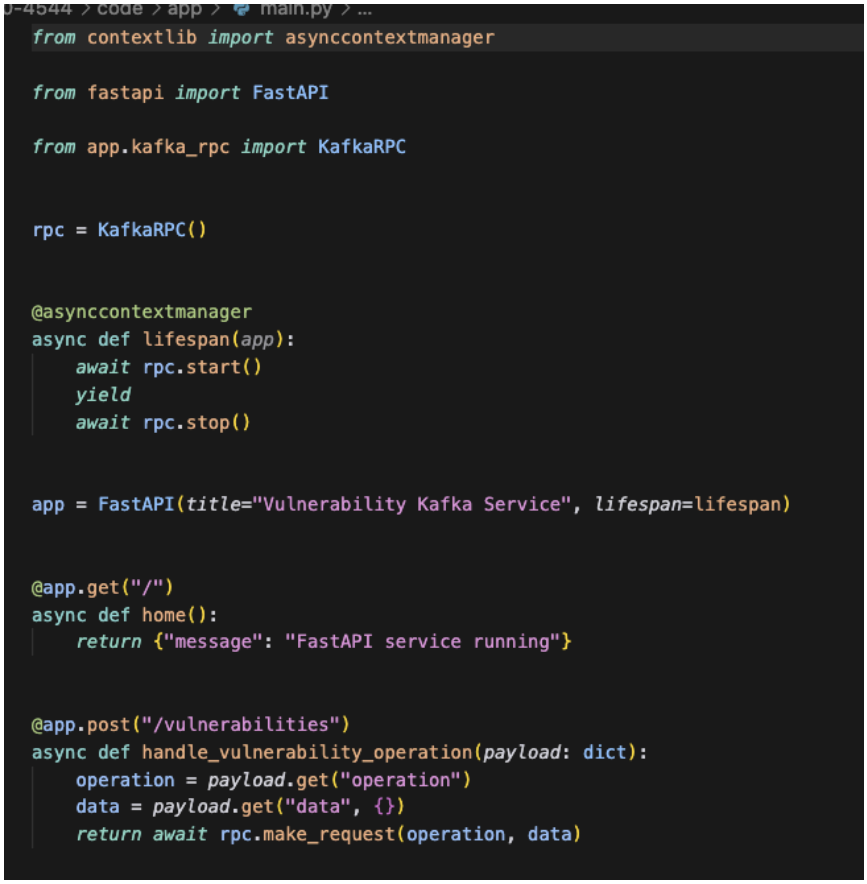
```
            future.set_result(response)

async def make_request(self, operation, data):
    correlation_id = str(uuid.uuid4())
    request = {
        "correlation_id": correlation_id,
        "reply_to": RESPONSE_TOPIC,
        "operation": operation,
        "data": data,
    }
    future = asyncio.get_event_loop().create_future()
    self.pending[correlation_id] = future

    print("[FASTAPI] Sending request to Kafka")
    print(f"[FASTAPI] topic={REQUEST_TOPIC}")
    print(f"[FASTAPI] request={json.dumps(request, indent=2)}")
    await self.producer.send_and_wait(
        REQUEST_TOPIC,
        json.dumps(request).encode(),
    )

    try:
        return await asyncio.wait_for(future, timeout=TIMEOUT_SECONDS)
    except asyncio.TimeoutError:
        self.pending.pop(correlation_id, None)
        return {
            "correlation_id": correlation_id,
            "success": False,
            "error": "Kafka response timeout",
        }
```

main.py



III. General Requirements

consumer.py

```python
def process_request_message(request):
    """Validate the Kafka envelope and return a response with its correlation ID."""
    correlation_id = request.get("correlation_id")

    if not correlation_id:
        return {
            "correlation_id": correlation_id,
            "success": False,
            "error": "correlation_id is required",
        }

    if not request.get("reply_to"):
        return {
            "correlation_id": correlation_id,
            "success": False,
            "error": "reply_to is required",
        }

    if not request.get("operation"):
        return {
            "correlation_id": correlation_id,
            "success": False,
            "error": "operation is required",
        }

    result = process_vulnerability_request(
        request["operation"], request.get("data")
    )
    return {"correlation_id": correlation_id, **result}
```

IV. Demonstration

homework_demo.py



V. Required Screenshots and Evidence

1.

```
(venv) (base) sangah@MacBook-Air-SAL data260-4544 % docker compose -f docker-compose.hw08.yml up -d --build

[+] up 12/12
 ✔ Image apache/kafka:3.9.2 Pulled
[+] Building 392.7s (15/15) FINISHED
 => [internal] load local bake definitions
 => => reading from stdin 1.03kB
 => [fastapi internal] load build definition from Dockerfile.hw08
 => => transferring dockerfile: 247B
 => [fastapi internal] load metadata for docker.io/library/python:3.12-slim
 => [auth] library/python:pull token for registry-1.docker.io
 => [fastapi internal] load .dockerignore
 => => transferring context: 2B
 => [worker internal] load build context
 => => transferring context: 70.46MB
 => [fastapi 1/5] FROM docker.io/library/python:3.12-slim@sha256:aa943ce411423c7582288f18707ff66eddcbaeb510502d747a72aeb34f30d159
 => => resolve docker.io/library/python:3.12-slim@sha256:aa943ce411423c7582288f18707ff66eddcbaeb510502d747a72aeb34f30d159
 => => sha256:64074ecd150a3526f7ef1af3128b9186744abfb9dc3d1c3e87c11c1ea880cbd0 251B / 251B
 => => sha256:b41621e272b0b306c196fabab6f5e0132d1c13f3c929bfb63455191fa509e311 12.06MB / 12.06MB
 => => sha256:0922d1fc1a5f7baa280015c744bb0be989e6e146a9181c04d2971e1273416844 1.27MB / 1.27MB
 => => sha256:2b70e040f7d132c8d366ee0223462552686510fb372ea5a83822973c30f19e36 30.21MB / 30.21MB
 => => extracting sha256:2b70e040f7d132c8d366ee0223462552686510fb372ea5a83822973c30f19e36
 => => extracting sha256:0922d1fc1a5f7baa280015c744bb0be989e6e146a9181c04d2971e1273416844
 => => extracting sha256:b41621e272b0b306c196fabab6f5e0132d1c13f3c929bfb63455191fa509e311
 => => extracting sha256:64074ecd150a3526f7ef1af3128b9186744abfb9dc3d1c3e87c11c1ea880cbd0
 => [worker 2/5] WORKDIR /workspace/code
 => [fastapi 3/5] COPY code/requirements-hw08.txt /workspace/code/requirements-hw08.txt
 => [worker 4/5] RUN pip install --no-cache-dir -r requirements-hw08.txt
 => [fastapi 5/5] COPY code/ /workspace/code/
 => [fastapi] exporting to image
 => => exporting layers
 => => exporting manifest sha256:5c592e2747f5e7e7730dc0bb429e58f649f2f66ce71671f9c0aaa59a4fe32220
 => => exporting config sha256:a145372bef90e0188c4218bf8633ecb198b8c4174afdbeaa50cad83582e9da72
 => => exporting attestation manifest sha256:35c77b89a1fa7e9e57d1ff6c328c85af164ed42e34ac4fe9d108146247fd3c77
 => => exporting manifest list sha256:ac041a5e74c77cc50fb5852fd569dcfc74114cce2d5550c75b1376be9e394be3
 => => naming to docker.io/library/data260-4544-fastapi:latest
 => => unpacking to docker.io/library/data260-4544-fastapi:latest
 => [worker] exporting to image
 => => exporting layers
 => => exporting manifest sha256:632c56f08a1caa305ef3474da12b709469758fb6c67d18e281450166d42ca451
 => => exporting config sha256:f0e91f0429ae6e1cd442ed2484792f5a9422e5589fb8dc18841f997b9529b022
 => => exporting attestation manifest sha256:54043e3e7c18ae0af041ccdd6117acaedb58239f40b4e55ea7f9b7c97ad72f63
 => => exporting manifest list sha256:1a6e1c99b8183707db61eecc6376e77bd40229c18ec57176d5f77ef0d07d2c2a
 => => naming to docker.io/library/data260-4544-worker:latest
 => => unpacking to docker.io/library/data260-4544-worker:latest
 => [fastapi] resolving provenance for metadata file
 => [worker] resolving provenance for metadata file
[+] up 18/18
 ✔ Image apache/kafka:3.9.2      Pulled
 ✔ Image data260-4544-fastapi    Built
 ✔ Image data260-4544-worker     Built
 ✔ Network data260-4544_default  Created
 ✔ Container s4544-kafka         Started
 ✔ Container s4544-worker        Started
 ✔ Container s4544-fastapi       Started
(venv) (base) sangah@MacBook-Air-SAL data260-4544 %
```

```
(venv) (base) sangah@MacBook-Air-SAL data260-4544 % docker compose -f docker-compose.hw08.yml ps
NAME           IMAGE                  COMMAND                  SERVICE   CREATED              STATUS          PORTS
s4544-fastapi  data260-4544-fastapi   "uvicorn app.main:ap…"   fastapi   About a minute ago   Up 59 seconds   0.0.0.0:8044->8044/tcp, [::]:8044->8044/tcp
s4544-kafka    apache/kafka:3.9.2     "/__cacert_entrypoin…"   kafka     28 minutes ago       Up 28 minutes   0.0.0.0:9092->9092/tcp, [::]:9092->9092/tcp
s4544-worker   data260-4544-worker    "python -u worker/co…"   worker    3 minutes ago        Up 3 minutes
(venv) (base) sangah@MacBook-Air-SAL data260-4544 %
```

```
 ✔ Container s4544-fastapi Started
(venv) (base) sangah@MacBook-Air-SAL data260-4544 % docker compose -f docker-compose.hw08.yml logs --tail=50 fastapi
s4544-fastapi  | INFO:     Started server process [1]
s4544-fastapi  | INFO:     Waiting for application startup.
s4544-fastapi  | Topic s4544_vulnerability_responses not found in cluster metadata
s4544-fastapi  | INFO:     Application startup complete.
s4544-fastapi  | INFO:     Uvicorn running on http://0.0.0.0:8044 (Press CTRL+C to quit)
(venv) (base) sangah@MacBook-Air-SAL data260-4544 %
```

```
(venv) (base) sangah@MacBook-Air-SAL data260-4544 % docker compose -f docker-compose.hw08.yml logs --tail=50 worker
s4544-worker  | The consumer is ready and listening...
s4544-worker  | Listening for requests on topic: s4544_vulnerability_requests
(venv) (base) sangah@MacBook-Air-SAL data260-4544 %
```

2.

```
s4544-worker    data260-4544-worker    "python -u worker/co…"   worker    3 minutes ago
(venv) (base) sangah@MacBook-Air-SAL data260-4544 % python code/homework_demo.py
<DEMO> correlation_id=fd6b181e-a62a-4b7e-be7e-f584c4eac4ea
<DEMO> Sending CREATE_VULNERABILITY request

[DEMO] CREATE_VULNERABILITY response
[DEMO] HTTP status=200
{
  "correlation_id": "fd6b181e-a62a-4b7e-be7e-f584c4eac4ea",
  "success": true,
  "entity_id": "1f20ca92-2f17-4714-a52e-f0b956ec1ccd",
  "message": "Entity created"
}
[DEMO] Generated entity_id=1f20ca92-2f17-4714-a52e-f0b956ec1ccd
<DEMO> Sending GET_VULNERABILITY request

[DEMO] GET_VULNERABILITY response
[DEMO] HTTP status=200
{
  "correlation_id": "fd6b181e-a62a-4b7e-be7e-f584c4eac4ea",
  "success": true,
  "entity": {
    "entity_id": "1f20ca92-2f17-4714-a52e-f0b956ec1ccd",
    "package_id": 1,
    "vulnerability_name": "Unsafe parser",
    "vulnerability_code": "CVE-DEMO-001",
    "urgency_score": 5
  }
}
<DEMO> correlation_id preserved successfully
<DEMO> CREATE_VULNERABILITY -> GET_VULNERABILITY complete
(venv) (base) sangah@MacBook-Air-SAL data260-4544 %
```

3.

```
<DEMO> CREATE_VULNERABILITY -> GET_VULNERABILITY complete
(venv) (base) sangah@MacBook-Air-SAL data260-4544 % docker compose -f docker-compose.hw08.yml logs --since=5m worker fastapi \
| grep -E "topic=|Processing|correlation_id"
s4544-fastapi  | [FASTAPI] topic=s4544_vulnerability_requests
s4544-fastapi  |   "correlation_id": "42c78a3d-ba8e-440f-9eb5-c79a8132efce",
s4544-worker   | [WORKER] Received request from topic=s4544_vulnerability_requests
s4544-worker   | [WORKER] correlation_id=42c78a3d-ba8e-440f-9eb5-c79a8132efce
s4544-worker   |   "correlation_id": "42c78a3d-ba8e-440f-9eb5-c79a8132efce",
s4544-worker   | [WORKER] Processing operation=CREATE_VULNERABILITY
s4544-worker   | [WORKER] Sending response to topic=s4544_vulnerability_responses
s4544-worker   |   "correlation_id": "42c78a3d-ba8e-440f-9eb5-c79a8132efce",
s4544-fastapi  | [FASTAPI] Sent request to topic=s4544_vulnerability_requests
s4544-fastapi  | [FASTAPI] Received response from topic=s4544_vulnerability_responses
s4544-fastapi  | [FASTAPI] correlation_id=42c78a3d-ba8e-440f-9eb5-c79a8132efce
s4544-fastapi  |   "correlation_id": "42c78a3d-ba8e-440f-9eb5-c79a8132efce",
s4544-worker   | [WORKER] Received request from topic=s4544_vulnerability_requests
s4544-worker   | [WORKER] correlation_id=42c78a3d-ba8e-440f-9eb5-c79a8132efce
s4544-worker   |   "correlation_id": "42c78a3d-ba8e-440f-9eb5-c79a8132efce",
s4544-worker   | [WORKER] Processing operation=GET_VULNERABILITY
s4544-worker   | [WORKER] Sending response to topic=s4544_vulnerability_responses
s4544-worker   |   "correlation_id": "42c78a3d-ba8e-440f-9eb5-c79a8132efce",
s4544-fastapi  | [FASTAPI] topic=s4544_vulnerability_requests
s4544-fastapi  |   "correlation_id": "42c78a3d-ba8e-440f-9eb5-c79a8132efce",
s4544-fastapi  | [FASTAPI] Sent request to topic=s4544_vulnerability_requests
s4544-fastapi  | [FASTAPI] Received response from topic=s4544_vulnerability_responses
s4544-fastapi  | [FASTAPI] correlation_id=42c78a3d-ba8e-440f-9eb5-c79a8132efce
s4544-fastapi  |   "correlation_id": "42c78a3d-ba8e-440f-9eb5-c79a8132efce",
(venv) (base) sangah@MacBook-Air-SAL data260-4544 %
```

4.

```
s4544-fastapi  | [FASTAPI] Returning response to caller
(base) sangah@MacBook-Air-SAL data260-4544 % docker compose -f docker-compose.hw08.yml logs \
--since=5m --timestamps --no-log-prefix worker fastapi \
| grep -E "Sending request|Received request|Processing|Sending response|Received response|Returning response|operation|topic=|correlation_id"
2026-10-30T00:10:55.937551631Z [WORKER] Received request from Kafka
2026-10-30T00:10:55.940414923Z [FASTAPI] Sending request to Kafka
2026-10-30T00:10:55.940415839Z [FASTAPI] topic=s4544_vulnerability_requests
2026-10-30T00:10:55.940417173Z   "correlation_id": "d3c09378-0855-464f-ba8c-7427fbe0855b",
2026-10-30T00:10:55.940418381Z   "operation": "CREATE_VULNERABILITY",
2026-10-30T00:10:55.940422881Z [FASTAPI] Sent request to topic=s4544_vulnerability_requests
2026-10-30T00:10:55.940423381Z [FASTAPI] Received response from topic=s4544_vulnerability_responses
2026-10-30T00:10:55.940423881Z [FASTAPI] correlation_id=d3c09378-0855-464f-ba8c-7427fbe0855b
2026-10-30T00:10:55.940425048Z   "correlation_id": "d3c09378-0855-464f-ba8c-7427fbe0855b",
2026-10-30T00:10:55.940427923Z [FASTAPI] Returning response to caller
2026-10-30T00:10:55.946692173Z [FASTAPI] Sending request to Kafka
2026-10-30T00:10:55.946699589Z [FASTAPI] topic=s4544_vulnerability_requests
2026-10-30T00:10:55.946701131Z   "correlation_id": "d3c09378-0855-464f-ba8c-7427fbe0855b",
2026-10-30T00:10:55.937621423Z [WORKER] Received request from topic=s4544_vulnerability_requests
2026-10-30T00:10:55.946702464Z   "operation": "GET_VULNERABILITY",
2026-10-30T00:10:55.937625089Z [WORKER] correlation_id=d3c09378-0855-464f-ba8c-7427fbe0855b
2026-10-30T00:10:55.937628214Z   "correlation_id": "d3c09378-0855-464f-ba8c-7427fbe0855b",
2026-10-30T00:10:55.937629673Z   "operation": "CREATE_VULNERABILITY",
2026-10-30T00:10:55.937638589Z [WORKER] Processing operation=CREATE_VULNERABILITY
2026-10-30T00:10:55.937639464Z [WORKER] Sending response to topic=s4544_vulnerability_responses
2026-10-30T00:10:55.937645089Z   "correlation_id": "d3c09378-0855-464f-ba8c-7427fbe0855b",
2026-10-30T00:10:55.945254048Z [WORKER] Received request from Kafka
2026-10-30T00:10:55.945289923Z [WORKER] Received request from topic=s4544_vulnerability_requests
2026-10-30T00:10:55.945293256Z [WORKER] correlation_id=d3c09378-0855-464f-ba8c-7427fbe0855b
2026-10-30T00:10:55.945297839Z   "correlation_id": "d3c09378-0855-464f-ba8c-7427fbe0855b",
2026-10-30T00:10:55.945300464Z   "operation": "GET_VULNERABILITY",
2026-10-30T00:10:55.945311006Z [WORKER] Processing operation=GET_VULNERABILITY
2026-10-30T00:10:55.945312631Z [WORKER] Sending response to topic=s4544_vulnerability_responses
2026-10-30T00:10:55.945315256Z   "correlation_id": "d3c09378-0855-464f-ba8c-7427fbe0855b",
2026-10-30T00:10:55.946705464Z [FASTAPI] Sent request to topic=s4544_vulnerability_requests
2026-10-30T00:10:55.946706214Z [FASTAPI] Received response from topic=s4544_vulnerability_responses
2026-10-30T00:10:55.946707006Z [FASTAPI] correlation_id=d3c09378-0855-464f-ba8c-7427fbe0855b
2026-10-30T00:10:55.946708006Z   "correlation_id": "d3c09378-0855-464f-ba8c-7427fbe0855b",
2026-10-30T00:10:55.946726423Z [FASTAPI] Returning response to caller
(base) sangah@MacBook-Air-SAL data260-4544 %
```

PART 2- Three-Agent Kafka Pipeline

I. Topics and Workflow

topics.py

```
-4544 > code > hw08_kafka_pipeline > topics.py > ...
# Shared Kafka topic names for the HW8 three-agent pipeline that is used for Part 2
PREFIX = "s4544"

INBOX_TOPIC = f"{PREFIX}_inbox"
TASKS_TOPIC = f"{PREFIX}_tasks"
DRAFTS_TOPIC = f"{PREFIX}_drafts"
FINAL_TOPIC = f"{PREFIX}_final"


PIPELINE = {
    "question": INBOX_TOPIC,
    "planner_output": TASKS_TOPIC,
    "writer_output": DRAFTS_TOPIC,
    "reviewer_output": FINAL_TOPIC,
}
```

II. Kafka Topic Creation

```
#!/bin/sh
KAFKA_CONTAINER="s4544-kafka"
BOOTSTRAP_SERVER="kafka:29092"

docker exec "$KAFKA_CONTAINER" /opt/kafka/bin/kafka-topics.sh \
  --create \
  --if-not-exists \
  --topic s4544_inbox \
  --partitions 1 \
  --replication-factor 1 \
  --bootstrap-server "$BOOTSTRAP_SERVER"

docker exec "$KAFKA_CONTAINER" /opt/kafka/bin/kafka-topics.sh \
  --create \
  --if-not-exists \
  --topic s4544_tasks \
  --partitions 2 \
  --replication-factor 1 \
  --bootstrap-server "$BOOTSTRAP_SERVER"

docker exec "$KAFKA_CONTAINER" /opt/kafka/bin/kafka-topics.sh \
  --create \
  --if-not-exists \
  --topic s4544_drafts \
  --partitions 1 \
  --replication-factor 1 \
  --bootstrap-server "$BOOTSTRAP_SERVER"

docker exec "$KAFKA_CONTAINER" /opt/kafka/bin/kafka-topics.sh \
  --create \
  --if-not-exists \
  --topic s4544_final \
  --partitions 1 \
  --replication-factor 1 \
  --bootstrap-server "$BOOTSTRAP_SERVER"

docker exec "$KAFKA_CONTAINER" /opt/kafka/bin/kafka-topics.sh \
  --list \
  --bootstrap-server "$BOOTSTRAP_SERVER"
```

```
zsh: permission denied: code/hw08_kafka_pipeline/create_topics.sh
(venv) (base) sangah@MacBook-Air-SAL data260-4544 % bash code/hw08_kafka_pipeline/create_topics.sh
WARNING: Due to limitations in metric names, topics with a period ('.') or underscore ('_') could collide. To avoid issues it is best to use either, but not both.
Created topic s4544_inbox.
WARNING: Due to limitations in metric names, topics with a period ('.') or underscore ('_') could collide. To avoid issues it is best to use either, but not both.
Created topic s4544_tasks.
WARNING: Due to limitations in metric names, topics with a period ('.') or underscore ('_') could collide. To avoid issues it is best to use either, but not both.
Created topic s4544_drafts.
WARNING: Due to limitations in metric names, topics with a period ('.') or underscore ('_') could collide. To avoid issues it is best to use either, but not both.
Created topic s4544_final.
__consumer_offsets
s4544_drafts
s4544_final
s4544_inbox
s4544_tasks
s4544_vulnerability_requests
s4544_vulnerability_responses
(venv) (base) sangah@MacBook-Air-SAL data260-4544 %
```

III. Running the Agents

planner.py

```python
# To run each agent in own terminal
import asyncio
import json
import os

from aiokafka import AIOKafkaConsumer, AIOKafkaProducer

from topics import INBOX_TOPIC, TASKS_TOPIC


BOOTSTRAP_SERVERS = os.getenv("KAFKA_BOOTSTRAP_SERVERS", "localhost:9092")


def make_plan(question):
    return [
        "Identify the main question",
        "Use the available vulnerability information",
        "Write a concise answer",
    ]


async def main():
    consumer = AIOKafkaConsumer(
        INBOX_TOPIC,
        bootstrap_servers=BOOTSTRAP_SERVERS,
        group_id="s4544-planner-group",
        auto_offset_reset="earliest",
        enable_auto_commit=True,
    )
    producer = AIOKafkaProducer(bootstrap_servers=BOOTSTRAP_SERVERS)

    await consumer.start()
    await producer.start()
    print(f"[PLANNER] Listening on {INBOX_TOPIC}")

    try:
        async for message in consumer:
            incoming = json.loads(message.value.decode())
            plan = make_plan(incoming["question"])
            outgoing = {
                "correlation_id": incoming["correlation_id"],
                "question": incoming["question"],
                "plan": plan,
            }
            print(f"[PLANNER] Received: {json.dumps(incoming)}")
            print(f"[PLANNER] Sending to {TASKS_TOPIC}: {json.dumps(outgoing)}")
            await producer.send_and_wait(TASKS_TOPIC, json.dumps(outgoing).encode())
    finally:
        await consumer.stop()
```

```
script: reports/hw08/RUN_LOG.txt: No such file or directory
(venv) (base) sangah@MacBook-Air-SAL hw08_kafka_pipeline % { date '+[%Y-%m-%d %H:%M:%S %Z] PLANNER'; python -u planner.py; } 2>&1 \
| tee -a ../../reports/hw08/RUN_LOG.txt
[2026-10-29 17:46:46 PDT] PLANNER
[PLANNER] Listening on s4544_inbox
[PLANNER] Received: {"correlation_id": "6851954d-e041-4bbd-ac65-3ae6704bb67b", "question": "How should a team prioritize a high-severity vulnerability?"}
[PLANNER] Sending to s4544_tasks: {"correlation_id": "6851954d-e041-4bbd-ac65-3ae6704bb67b", "question": "How should a team prioritize a high-severity vulnerability?", "plan":
 ["Identify the main question", "Use the available vulnerability information", "Write a concise answer"]}
```

writer.py

```python
# To run each agent in own terminal
import asyncio
import json
import os

from aiokafka import AIOKafkaConsumer, AIOKafkaProducer

from topics import DRAFTS_TOPIC, TASKS_TOPIC


BOOTSTRAP_SERVERS = os.getenv("KAFKA_BOOTSTRAP_SERVERS", "localhost:9092")


def write_draft(question, plan):
    return (
        f"The question was: {question} "
        f"The response follows this plan: {', '.join(plan)}."
    )


async def main():
    consumer = AIOKafkaConsumer(
        TASKS_TOPIC,
        bootstrap_servers=BOOTSTRAP_SERVERS,
        group_id="s4544-writer-group",
        auto_offset_reset="earliest",
        enable_auto_commit=True,
    )
    producer = AIOKafkaProducer(bootstrap_servers=BOOTSTRAP_SERVERS)

    await consumer.start()
    await producer.start()
    print(f"[WRITER] Listening on {TASKS_TOPIC}")

    try:
        async for message in consumer:
            incoming = json.loads(message.value.decode())
            outgoing = {
                "correlation_id": incoming["correlation_id"],
                "question": incoming["question"],
                "plan": incoming["plan"],
                "draft": write_draft(incoming["question"], incoming["plan"]),
            }
            print(f"[WRITER] Received: {json.dumps(incoming)}")
            print(f"[WRITER] Sending to {DRAFTS_TOPIC}: {json.dumps(outgoing)}")
            await producer.send_and_wait(DRAFTS_TOPIC, json.dumps(outgoing).encode())
    finally:
        await consumer.stop()
        await producer.stop()
```

```
(venv) (base) sangah@MacBook-Air-SAL hw08_kafka_pipeline % { date '+[%Y-%m-%d %H:%M:%S %Z] WRITER'; python -u writer.py; } 2>&1 \
| tee -a ../../reports/hw08/RUN_LOG.txt
[2026-10-29 17:48:16 PDT] WRITER
[WRITER] Listening on s4544_tasks
[WRITER] Received: {"correlation_id": "6851954d-e041-4bbd-ac65-3ae6704bb67b", "question": "How should a team prioritize a high-severity vulnerability?", "plan": ["Identify the
 main question", "Use the available vulnerability information", "Write a concise answer"]}
[WRITER] Sending to s4544_drafts: {"correlation_id": "6851954d-e041-4bbd-ac65-3ae6704bb67b", "question": "How should a team prioritize a high-severity vulnerability?", "plan":
 ["Identify the main question", "Use the available vulnerability information", "Write a concise answer"], "draft": "The question was: How should a team prioritize a high-sever
ity vulnerability? The response follows this plan: Identify the main question, Use the available vulnerability information, Write a concise answer."}
```

reviewer.py

```python
# To run each agent in own terminal
import asyncio
import json
import os

from aiokafka import AIOKafkaConsumer, AIOKafkaProducer

from topics import DRAFTS_TOPIC, FINAL_TOPIC


BOOTSTRAP_SERVERS = os.getenv("KAFKA_BOOTSTRAP_SERVERS", "localhost:9092")


def review_draft(draft):
    if not draft.strip():
        return {"status": "rejected", "answer": "Draft was empty"}
    return {"status": "approved", "answer": draft}


async def main():
    consumer = AIOKafkaConsumer(
        DRAFTS_TOPIC,
        bootstrap_servers=BOOTSTRAP_SERVERS,
        group_id="s4544-reviewer-group",
        auto_offset_reset="earliest",
        enable_auto_commit=True,
    )
    producer = AIOKafkaProducer(bootstrap_servers=BOOTSTRAP_SERVERS)

    await consumer.start()
    await producer.start()
    print(f"[REVIEWER] Listening on {DRAFTS_TOPIC}")

    try:
        async for message in consumer:
            incoming = json.loads(message.value.decode())
            review = review_draft(incoming["draft"])
            outgoing = {
                "correlation_id": incoming["correlation_id"],
                "status": review["status"],
                "answer": review["answer"],
            }
            print(f"[REVIEWER] Received: {json.dumps(incoming)}")
            print(f"[REVIEWER] Sending to {FINAL_TOPIC}: {json.dumps(outgoing)}")
            await producer.send_and_wait(FINAL_TOPIC, json.dumps(outgoing).encode())
    finally:
        await consumer.stop()
        await producer.stop()
```

```
__pycache__             planner.py              send_question.py        writer.py
(venv) (base) sangah@MacBook-Air-SAL hw08_kafka_pipeline % { date '+[%Y-%m-%d %H:%M:%S %Z] REVIEWER'; python -u reviewer.py; } 2>&1 \
| tee -a ../../reports/hw08/RUN_LOG.txt
[2026-10-29 17:48:57 PDT] REVIEWER
[REVIEWER] Listening on s4544_drafts
[REVIEWER] Received: {"correlation_id": "6851954d-e041-4bbd-ac65-3ae6704bb67b", "question": "How should a team prioritize a high-severity vulnerability?", "plan": ["Identify t
he main question", "Use the available vulnerability information", "Write a concise answer"], "draft": "The question was: How should a team prioritize a high-severity vulnerabi
lity? The response follows this plan: Identify the main question, Use the available vulnerability information, Write a concise answer."}
[REVIEWER] Sending to s4544_final: {"correlation_id": "6851954d-e041-4bbd-ac65-3ae6704bb67b", "status": "approved", "answer": "The question was: How should a team prioritize a
 high-severity vulnerability? The response follows this plan: Identify the main question, Use the available vulnerability information, Write a concise answer."}
```

send_question.py

```
0-4544 > code > hw08_kafka_pipeline > send_question.py > ...
# To run each agent in own terminal
import asyncio
import json
import os
import uuid

from aiokafka import AIOKafkaConsumer, AIOKafkaProducer

from topics import FINAL_TOPIC, INBOX_TOPIC


BOOTSTRAP_SERVERS = os.getenv("KAFKA_BOOTSTRAP_SERVERS", "localhost:9092")


async def main():
    correlation_id = str(uuid.uuid4())
    question = "How should a team prioritize a high-severity vulnerability?"
    message = {
        "correlation_id": correlation_id,
        "question": question,
    }

    producer = AIOKafkaProducer(bootstrap_servers=BOOTSTRAP_SERVERS)
    consumer = AIOKafkaConsumer(
        FINAL_TOPIC,
        bootstrap_servers=BOOTSTRAP_SERVERS,
        group_id=f"s4544-final-reader-{correlation_id}",
        auto_offset_reset="latest",
        enable_auto_commit=True,
    )

    await producer.start()
    await consumer.start()
    print(f"[SENDER] Sending to {INBOX_TOPIC}: {json.dumps(message)}")
    await producer.send_and_wait(INBOX_TOPIC, json.dumps(message).encode())

    try:
        async for record in consumer:
            result = json.loads(record.value.decode())
            if result.get("correlation_id") == correlation_id:
                print(f"[SENDER] Final result from {FINAL_TOPIC}:")
                print(json.dumps(result, indent=2))
                break
    finally:
        await consumer.stop()
        await producer.stop()


if __name__ == "__main__":
    asyncio.run(main())
```

```
(venv) (base) sangah@MacBook-Air-SAL code % cd hw08_kafka_pipeline
(venv) (base) sangah@MacBook-Air-SAL hw08_kafka_pipeline % { date '+[%Y-%m-%d %H:%M:%S %Z] SENDER'; python -u send_question.py; } 2>&1 \
| tee -a ../../reports/hw08/RUN_LOG.txt
[2026-10-29 17:49:56 PDT] SENDER
[SENDER] Sending to s4544_inbox: {"correlation_id": "6851954d-e041-4bbd-ac65-3ae6704bb67b", "question": "How should a team prioritize a high-severity vulnerability?"}
[SENDER] Final result from s4544_final:
{
  "correlation_id": "6851954d-e041-4bbd-ac65-3ae6704bb67b",
  "status": "approved",
  "answer": "The question was: How should a team prioritize a high-severity vulnerability? The response follows this plan: Identify the main question, Use the available vulner
ability information, Write a concise answer."
}
(venv) (base) sangah@MacBook-Air-SAL hw08_kafka_pipeline %
```

IV. Result

RUN_LOG.txt

```
Script done on Thu Oct 29 17:44:54 2026
[2026-10-29 17:46:46 PDT] PLANNER
[PLANNER] Listening on s4544_inbox
[2026-10-29 17:48:16 PDT] WRITER
[WRITER] Listening on s4544_tasks
[2026-10-29 17:48:57 PDT] REVIEWER
[REVIEWER] Listening on s4544_drafts
[2026-10-29 17:49:56 PDT] SENDER
[SENDER] Sending to s4544_inbox: {"correlation_id": "6851954d-e041-4bbd-ac65-3ae6704bb67b", "question": "How should a team prioritize a high-severity vulnerability?"}
[PLANNER] Received: {"correlation_id": "6851954d-e041-4bbd-ac65-3ae6704bb67b", "question": "How should a team prioritize a high-severity vulnerability?"}
[PLANNER] Sending to s4544_tasks: {"correlation_id": "6851954d-e041-4bbd-ac65-3ae6704bb67b", "question": "How should a team prioritize a high-severity vulnerability?", "plan": ["Identify the main question", "Us
[WRITER] Received: {"correlation_id": "6851954d-e041-4bbd-ac65-3ae6704bb67b", "question": "How should a team prioritize a high-severity vulnerability?", "plan": ["Identify the main question", "Use the available
[WRITER] Sending to s4544_drafts: {"correlation_id": "6851954d-e041-4bbd-ac65-3ae6704bb67b", "question": "How should a team prioritize a high-severity vulnerability?", "plan": ["Identify the main question", "Us
[REVIEWER] Received: {"correlation_id": "6851954d-e041-4bbd-ac65-3ae6704bb67b", "question": "How should a team prioritize a high-severity vulnerability?", "plan": ["Identify the main question", "Use the availat
[REVIEWER] Sending to s4544_final: {"correlation_id": "6851954d-e041-4bbd-ac65-3ae6704bb67b", "status": "approved", "answer": "The question was: How should a team prioritize a high-severity vulnerability? The r
[SENDER] Final result from s4544_final:
{
  "correlation_id": "6851954d-e041-4bbd-ac65-3ae6704bb67b",
  "status": "approved",
  "answer": "The question was: How should a team prioritize a high-severity vulnerability? The response follows this plan: Identify the main question, Use the available vulnerability information, Write a concis
}
```

For Homework 2, Planner, Writer, and Reviewer were function calls inside one process, so each step directly waited for the previous one. However, in HW8, each agent runs as a separate process and communicates through Kafka topics, allowing more independence. The agents were able to scale independently, and one agent can fail or restart without stopping the entire pipeline because Kafka stores messages and the correlation_id connects each stage. The agents acted because there was Kafka as the message system between them. After the sender has the question placed in the inbox, the Planner reads it and publishes a plan to tasks, and then the Writer reads the plan and publishes it to drafts. The Reviewer was then able to read the draft and publish the approved result to final.

PART 3 - Failure Behavior

I. Duplicate Delivery

replay_duplicate.py



```
(venv) (base) sangah@MacBook-Air-SAL hw08_kafka_pipeline % cd /Users/sangah/Desktop/Data260/data260-4544/code/hw08_kafka_pipeline

{ date '+[%Y-%m-%d %H:%M:%S %Z] DUPLICATE WRITER'; python -u writer.py; } 2>&1 \
| tee -a ../../reports/hw08/RUN_LOG.txt
[2026-10-29 19:21:41 PDT] DUPLICATE WRITER
[WRITER] Listening on s4544_tasks
[WRITER] Received: {"correlation_id": "duplicate-demo-0aef7989-d7b0-4aaa-b263-10519f805463", "question": "How should a team prioritize a high-severity vulnerability?", "plan":
 ["Identify the main question", "Use the available vulnerability information", "Write a concise answer"]}
[WRITER] Sending to s4544_drafts: {"correlation_id": "duplicate-demo-0aef7989-d7b0-4aaa-b263-10519f805463", "question": "How should a team prioritize a high-severity vulnerabi
lity?", "plan": ["Identify the main question", "Use the available vulnerability information", "Write a concise answer"], "draft": "The question was: How should a team prioriti
ze a high-severity vulnerability? The response follows this plan: Identify the main question, Use the available vulnerability information, Write a concise answer."}
[WRITER] Duplicate correlation_id ignored: duplicate-demo-0aef7989-d7b0-4aaa-b263-10519f805463
```

```
(venv) (base) sangah@MacBook-Air-SAL hw08_kafka_pipeline % { date '+[%Y-%m-%d %H:%M:%S %Z] DUPLICATE REPLAY'; python -u replay_duplicate.py; } 2>&1 \
| tee -a ../../reports/hw08/RUN_LOG.txt
[2026-10-29 19:21:46 PDT] DUPLICATE REPLAY
[REPLAY] Sending attempt 1 with correlation_id=duplicate-demo-0aef7989-d7b0-4aaa-b263-10519f805463
[REPLAY] Sending attempt 2 with correlation_id=duplicate-demo-0aef7989-d7b0-4aaa-b263-10519f805463
(venv) (base) sangah@MacBook-Air-SAL hw08_kafka_pipeline %
```

II. Consumer Death

writer.py

```python
    producer = AIOKafkaProducer(bootstrap_servers=BOOTSTRAP_SERVERS)

    await consumer.start()
    await producer.start()
    print(f"[WRITER] Listening on {TASKS_TOPIC}")

    try:
        async for message in consumer:
            incoming = json.loads(message.value.decode())
            correlation_id = incoming["correlation_id"]
            if correlation_id in processed_correlation_ids:
                print(
                    f"[WRITER] Duplicate correlation_id ignored: "
                    f"{correlation_id}"
                )
                continue

            if CRASH_AFTER_RECEIVE:
                print(
                    f"[WRITER] Simulating consumer death before commit for "
                    f"{correlation_id}"
                )
                raise RuntimeError("Simulated writer failure")

            processed_correlation_ids.add(correlation_id)
            outgoing = {
                "correlation_id": correlation_id,
                "question": incoming["question"],
                "plan": incoming["plan"],
                "draft": write_draft(incoming["question"], incoming["plan"]),
            }
            print(f"[WRITER] Received: {json.dumps(incoming)}")
            print(f"[WRITER] Sending to {DRAFTS_TOPIC}: {json.dumps(outgoing)}")
            await producer.send_and_wait(DRAFTS_TOPIC, json.dumps(outgoing).encode())
            await consumer.commit()
            print(f"[WRITER] Offset committed for {correlation_id}")
    finally:
        await consumer.stop()
        await producer.stop()
```

consumer_death_test.py

data260-4544 > code > hw08_kafka_pipeline > consumer_death_test.py > ...

```python
import asyncio
import json
import os

from aiokafka import AIOKafkaProducer

from topics import TASKS_TOPIC


BOOTSTRAP_SERVERS = os.getenv("KAFKA_BOOTSTRAP_SERVERS", "localhost:9092")


async def main():
    message = {
        "correlation_id": "consumer-death-demo-001",
        "question": "What should happen when a writer stops during processing?",
        "plan": [
            "Detect the failed consumer",
            "Redeliver the uncommitted message",
            "Commit after successful processing",
        ],
    }

    producer = AIOKafkaProducer(bootstrap_servers=BOOTSTRAP_SERVERS)
    await producer.start()
    try:
        print(f"[DEATH TEST] Sending to {TASKS_TOPIC}")
        print(json.dumps(message, indent=2))
        await producer.send_and_wait(
            TASKS_TOPIC,
            json.dumps(message).encode(),
            key=message["correlation_id"].encode(),
        )
    finally:
        await producer.stop()


if __name__ == "__main__":
    asyncio.run(main())
```

```
^C
(venv) (base) sangah@MacBook-Air-SAL hw08_kafka_pipeline % { date '+[%Y-%m-%d %H:%M:%S %Z] CONSUMER DEATH'; \
CRASH_AFTER_RECEIVE=1 python -u writer.py; } 2>&1 \
| tee -a ../../reports/hw08/RUN_LOG.txt
[2026-10-29 19:26:41 PDT] CONSUMER DEATH
[WRITER] Listening on s4544_tasks
[WRITER] Simulating consumer death before commit for consumer-death-demo-001
Traceback (most recent call last):
  File "/Users/sangah/Desktop/Data260/data260-4544/code/hw08_kafka_pipeline/writer.py", line 74, in <module>
    asyncio.run(main())
    ^^^^^^^^^^^^^^^^^^^
  File "/opt/anaconda3/lib/python3.13/asyncio/runners.py", line 195, in run
    return runner.run(main)
           ^^^^^^^^^^^^^^^^
  File "/opt/anaconda3/lib/python3.13/asyncio/runners.py", line 118, in run
    return self._loop.run_until_complete(task)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/opt/anaconda3/lib/python3.13/asyncio/base_events.py", line 725, in run_until_complete
    return future.result()
           ^^^^^^^^^^^^^^^
  File "/Users/sangah/Desktop/Data260/data260-4544/code/hw08_kafka_pipeline/writer.py", line 54, in main
    raise RuntimeError("Simulated writer failure")
RuntimeError: Simulated writer failure
(venv) (base) sangah@MacBook-Air-SAL hw08_kafka_pipeline % { date '+[%Y-%m-%d %H:%M:%S %Z] WRITER RESTART'; \
python -u writer.py; } 2>&1 \
| tee -a ../../reports/hw08/RUN_LOG.txt
[2026-10-29 19:27:43 PDT] WRITER RESTART
[WRITER] Listening on s4544_tasks
[WRITER] Received: {"correlation_id": "consumer-death-demo-001", "question": "What should happen when a writer stops during processing?", "plan": ["Detect the failed consumer"
, "Redeliver the uncommitted message", "Commit after successful processing"]}
[WRITER] Sending to s4544_drafts: {"correlation_id": "consumer-death-demo-001", "question": "What should happen when a writer stops during processing?", "plan": ["Detect the f
ailed consumer", "Redeliver the uncommitted message", "Commit after successful processing"], "draft": "The question was: What should happen when a writer stops during processi
ng? The response follows this plan: Detect the failed consumer, Redeliver the uncommitted message, Commit after successful processing."}
[WRITER] Offset committed for consumer-death-demo-001
```

```
(venv) (base) sangah@MacBook-Air-SAL hw08_kafka_pipeline % { date '+[%Y-%m-%d %H:%M:%S %Z] DEATH TEST'; \
python -u consumer_death_test.py; } 2>&1 \
| tee -a ../../reports/hw08/RUN_LOG.txt
[2026-10-29 19:26:49 PDT] DEATH TEST
[DEATH TEST] Sending to s4544_tasks
{
  "correlation_id": "consumer-death-demo-001",
  "question": "What should happen when a writer stops during processing?",
  "plan": [
    "Detect the failed consumer",
    "Redeliver the uncommitted message",
    "Commit after successful processing"
  ]
}
(venv) (base) sangah@MacBook-Air-SAL hw08_kafka_pipeline % { date '+[%Y-%m-%d %H:%M:%S %Z] DEATH TEST'; \
python -u consumer_death_test.py; } 2>&1 \
| tee -a ../../reports/hw08/RUN_LOG.txt
[2026-10-29 19:28:50 PDT] DEATH TEST
[DEATH TEST] Sending to s4544_tasks
{
  "correlation_id": "consumer-death-demo-001",
  "question": "What should happen when a writer stops during processing?",
  "plan": [
    "Detect the failed consumer",
    "Redeliver the uncommitted message",
    "Commit after successful processing"
  ]
}
```

III. Poison Message

poison_message_test.py

a260-4544 > code > hw08_kafka_pipeline > poison_message_test.py > main

```python
import asyncio
import json
import os

from aiokafka import AIOKafkaProducer

from topics import TASKS_TOPIC


BOOTSTRAP_SERVERS = os.getenv("KAFKA_BOOTSTRAP_SERVERS", "localhost:9092")
TEST_KEY = b"poison-order-test"


async def main():
    poison = {
        "correlation_id": "poison-demo-001",
        "question": "",
        "plan": [],
    }
    valid = {
        "correlation_id": "after-poison-001",
        "question": "What should happen after a poison message?",
        "plan": ["Retry the bad message", "Move it to dead letter", "Process later messages"],
    }

    producer = AIOKafkaProducer(bootstrap_servers=BOOTSTRAP_SERVERS)
    await producer.start()
    try:
        print(f"[POISON TEST] Sending poison message to {TASKS_TOPIC}")
        await producer.send_and_wait(
            TASKS_TOPIC, json.dumps(poison).encode(), key=TEST_KEY
        )
        print(f"[POISON TEST] Sending valid message after poison message")
        await producer.send_and_wait(
            TASKS_TOPIC, json.dumps(valid).encode(), key=TEST_KEY
        )
    finally:
        await producer.stop()
```

create_topics.sh

```
docker exec "$KAFKA_CONTAINER" /opt/kafka/bin/kafka-topics.sh \
  --create \
  --if-not-exists \
  --topic s4544_dead_letter \
  --partitions 1 \
  --replication-factor 1 \
  --bootstrap-server "$BOOTSTRAP_SERVER"
```

writer.py

```
async def main():
    consumer = AIOKafkaConsumer(
        TASKS_TOPIC,
        bootstrap_servers=BOOTSTRAP_SERVERS,
        group_id=WRITER_GROUP_ID,
        auto_offset_reset="earliest",
        enable_auto_commit=False,
    )
```

```
async def main():
    try:
        async for message in consumer:
            incoming = json.loads(message.value.decode())
            validation_error = validate_task(incoming)
            if validation_error:
                retry_count = incoming.get("retry_count", 0)
                if retry_count < MAX_RETRIES:
                    retry_message = {
                        **incoming,
                        "retry_count": retry_count + 1,
                        "error": validation_error,
                    }
                    print(
                        f"[WRITER] Poison message retry "
                        f"{retry_count + 1}/{MAX_RETRIES}: "
                        f"{validation_error}"
                    )
                    await producer.send_and_wait(
                        TASKS_TOPIC,
                        json.dumps(retry_message).encode(),
                        key=str(incoming.get("correlation_id", "poison")).encode(),
                    )
                else:
                    dead_letter = {
                        **incoming,
                        "dead_letter": True,
                        "error": validation_error,
                    }
                    print(
                        f"[WRITER] Moving poison message to "
                        f"{DEAD_LETTER_TOPIC} after {MAX_RETRIES} retries"
                    )
                    await producer.send_and_wait(
                        DEAD_LETTER_TOPIC,
                        json.dumps(dead_letter).encode(),
                    )
                    await consumer.commit()
                    continue
```

```
                    json.dumps(dead_letter).encode(),
                )
            await consumer.commit()
            continue

        correlation_id = incoming["correlation_id"]
        if correlation_id in processed_correlation_ids:
            print(
                f"[WRITER] Duplicate correlation_id ignored: "
                f"{correlation_id}"
            )
            continue

        if CRASH_AFTER_RECEIVE:
            print(
                f"[WRITER] Simulating consumer death before commit for "
                f"{correlation_id}"
            )
            raise RuntimeError("Simulated writer failure")

        processed_correlation_ids.add(correlation_id)
        outgoing = {
            "correlation_id": correlation_id,
            "question": incoming["question"],
            "plan": incoming["plan"],
            "draft": write_draft(incoming["question"], incoming["plan"]),
        }
        print(f"[WRITER] Received: {json.dumps(incoming)}")
        print(f"[WRITER] Sending to {DRAFTS_TOPIC}: {json.dumps(outgoing)}")
        await producer.send_and_wait(DRAFTS_TOPIC, json.dumps(outgoing).encode())
        await consumer.commit()
        print(f"[WRITER] Offset committed for {correlation_id}")
    finally:
```

```
(venv) (base) sangah@MacBook-Air-SAL hw08_kafka_pipeline % bash create_topics.sh
WARNING: Due to limitations in metric names, topics with a period ('.') or underscore ('_') could collide. To avoid issues it is best to use either, but not both.
WARNING: Due to limitations in metric names, topics with a period ('.') or underscore ('_') could collide. To avoid issues it is best to use either, but not both.
Created topic s4544_dead_letter.
WARNING: Due to limitations in metric names, topics with a period ('.') or underscore ('_') could collide. To avoid issues it is best to use either, but not both.
WARNING: Due to limitations in metric names, topics with a period ('.') or underscore ('_') could collide. To avoid issues it is best to use either, but not both.
WARNING: Due to limitations in metric names, topics with a period ('.') or underscore ('_') could collide. To avoid issues it is best to use either, but not both.
__consumer_offsets
s4544_dead_letter
s4544_drafts
s4544_final
s4544_inbox
s4544_tasks
s4544_vulnerability_requests
s4544_vulnerability_responses
(venv) (base) sangah@MacBook-Air-SAL hw08_kafka_pipeline %
```

```
(venv) (base) sangah@MacBook-Air-SAL hw08_kafka_pipeline % python -u writer.py
[WRITER] Listening on s4544_tasks
[WRITER] Poison message retry 1/3: question is required
[WRITER] Received: {"correlation_id": "after-poison-001", "question": "What should happen after a poison message?", "plan": ["Retry the bad message", "Move it to dead letter",
 "Process later messages"]}
[WRITER] Sending to s4544_drafts: {"correlation_id": "after-poison-001", "question": "What should happen after a poison message?", "plan": ["Retry the bad message", "Move it t
o dead letter", "Process later messages"], "draft": "The question was: What should happen after a poison message? The response follows this plan: Retry the bad message, Move i
t to dead letter, Process later messages."}
[WRITER] Offset committed for after-poison-001
[WRITER] Poison message retry 2/3: question is required
[WRITER] Poison message retry 3/3: question is required
[WRITER] Moving poison message to s4544_dead_letter after 3 retries
```

```
(venv) (base) sangah@MacBook-Air-SAL hw08_kafka_pipeline % python -u poison_message_test.py
[POISON TEST] Sending poison message to s4544_tasks
[POISON TEST] Sending valid message after poison message
(venv) (base) sangah@MacBook-Air-SAL hw08_kafka_pipeline %
```

IV. Ordering

writer.py



```
        continue

    correlation_id = incoming["correlation_id"]
    if correlation_id in processed_correlation_ids:
        print(
            f"[WRITER-{WRITER_INSTANCE}] Duplicate correlation_id "
            f"ignored: {correlation_id}"
        )
        continue

    if CRASH_AFTER_RECEIVE:
        print(
            f"[WRITER-{WRITER_INSTANCE}] Simulating consumer death before commit for "
            f"{correlation_id}"
        )
        raise RuntimeError("Simulated writer failure")

    processed_correlation_ids.add(correlation_id)
    outgoing = {
        "correlation_id": correlation_id,
        "question": incoming["question"],
        "plan": incoming["plan"],
        "draft": write_draft(incoming["question"], incoming["plan"]),
    }
    print(
        f"[WRITER-{WRITER_INSTANCE}] Received partition="
        f"{message.partition} offset={message.offset} "
        f"sequence={incoming.get('sequence')} "
        f"correlation_id={correlation_id}"
    )
    print(
        f"[WRITER-{WRITER_INSTANCE}] Sending to {DRAFTS_TOPIC}: "
        f"{json.dumps(outgoing)}"
    )
```

ordering_test.py

4544 > code > hw08_kafka_pipeline > ordering_test.py > ...

```python
import asyncio
import json
import os
import uuid

from aiokafka import AIOKafkaProducer

from topics import TASKS_TOPIC


BOOTSTRAP_SERVERS = os.getenv("KAFKA_BOOTSTRAP_SERVERS", "localhost:9092")
KEYED = os.getenv("ORDERING_KEYED") == "1"
ENTITY_ID = "related-entity-4544"


async def main():
    producer = AIOKafkaProducer(bootstrap_servers=BOOTSTRAP_SERVERS)
    await producer.start()
    mode = "keyed" if KEYED else "unkeyed"
    print(f"[ORDERING] Starting {mode} run with 20 messages")

    try:
        for sequence in range(1, 21):
            message = {
                "correlation_id": f"ordering-{mode}-{uuid.uuid4()}",
                "entity_id": ENTITY_ID,
                "sequence": sequence,
                "question": f"Ordering message {sequence}",
                "plan": ["Preserve related message order"],
            }
            key = ENTITY_ID.encode() if KEYED else None
            await producer.send_and_wait(
                TASKS_TOPIC,
                json.dumps(message).encode(),
                key=key,
            )
            print(
                f"[ORDERING] Sent sequence={sequence} "
                f"keyed={KEYED}"
            )
    finally:
        await producer.stop()


if __name__ == "__main__":
    asyncio.run(main())
```

```
(venv) (base) sangah@MacBook-Air-SAL hw08_kafka_pipeline % { date '+[%Y-%m-%d %H:%M:%S %Z] ORDERING WRITER 1'; \
  WRITER_INSTANCE=one python -u writer.py; } 2>&1 \
  | tee -a ../../reports/hw08/RUN_LOG.txt
  [2026-10-29 19:43:33 PDT] ORDERING WRITER 1
  [WRITER-one] Listening on s4544_tasks
  Heartbeat failed for group s4544-writer-group because it is rebalancing
  [WRITER-one] Received partition=0 offset=8 sequence=2 correlation_id=ordering-unkeyed-c346a735-58cb-4d0e-bdd8-700172fd5844
  [WRITER-one] Sending to s4544_drafts: {"correlation_id": "ordering-unkeyed-c346a735-58cb-4d0e-bdd8-700172fd5844", "question": "Ordering message 2", "plan": ["Preserve related
  message order"], "draft": "The question was: Ordering message 2 The response follows this plan: Preserve related message order."}
  [WRITER-one] Offset committed for ordering-unkeyed-c346a735-58cb-4d0e-bdd8-700172fd5844
  [WRITER-one] Received partition=0 offset=9 sequence=4 correlation_id=ordering-unkeyed-ac6a404d-f4ca-40e4-a1d8-0a7477fcfafd
  [WRITER-one] Sending to s4544_drafts: {"correlation_id": "ordering-unkeyed-ac6a404d-f4ca-40e4-a1d8-0a7477fcfafd", "question": "Ordering message 4", "plan": ["Preserve related
  message order"], "draft": "The question was: Ordering message 4 The response follows this plan: Preserve related message order."}
  [WRITER-one] Offset committed for ordering-unkeyed-ac6a404d-f4ca-40e4-a1d8-0a7477fcfafd
  [WRITER-one] Received partition=0 offset=10 sequence=5 correlation_id=ordering-unkeyed-2ed7ebb2-8c76-4c33-a12d-b2b55e09f47b
  [WRITER-one] Sending to s4544_drafts: {"correlation_id": "ordering-unkeyed-2ed7ebb2-8c76-4c33-a12d-b2b55e09f47b", "question": "Ordering message 5", "plan": ["Preserve related
  message order"], "draft": "The question was: Ordering message 5 The response follows this plan: Preserve related message order."}
  [WRITER-one] Offset committed for ordering-unkeyed-2ed7ebb2-8c76-4c33-a12d-b2b55e09f47b
  [WRITER-one] Received partition=0 offset=11 sequence=7 correlation_id=ordering-unkeyed-97ec64a0-3d44-4c25-b75b-ba47abac6b06
  [WRITER-one] Sending to s4544_drafts: {"correlation_id": "ordering-unkeyed-97ec64a0-3d44-4c25-b75b-ba47abac6b06", "question": "Ordering message 7", "plan": ["Preserve related
  message order"], "draft": "The question was: Ordering message 7 The response follows this plan: Preserve related message order."}
  [WRITER-one] Offset committed for ordering-unkeyed-97ec64a0-3d44-4c25-b75b-ba47abac6b06
  [WRITER-one] Received partition=0 offset=12 sequence=8 correlation_id=ordering-unkeyed-e85632a4-11a4-444d-9f0b-4f2a048c3cf4
  [WRITER-one] Sending to s4544_drafts: {"correlation_id": "ordering-unkeyed-e85632a4-11a4-444d-9f0b-4f2a048c3cf4", "question": "Ordering message 8", "plan": ["Preserve related
  message order"], "draft": "The question was: Ordering message 8 The response follows this plan: Preserve related message order."}
  [WRITER-one] Offset committed for ordering-unkeyed-e85632a4-11a4-444d-9f0b-4f2a048c3cf4
  [WRITER-one] Received partition=0 offset=13 sequence=10 correlation_id=ordering-unkeyed-34daf02c-5cc6-406c-8cb7-915926b94716
  [WRITER-one] Sending to s4544_drafts: {"correlation_id": "ordering-unkeyed-34daf02c-5cc6-406c-8cb7-915926b94716", "question": "Ordering message 10", "plan": ["Preserve relate
   message order"], "draft": "The question was: Ordering message 10 The response follows this plan: Preserve related message order."}
  [WRITER-one] Offset committed for ordering-unkeyed-34daf02c-5cc6-406c-8cb7-915926b94716
```

```
  [WRITER-one] Sending to s4544_drafts: {"correlation_id": "ordering-keyed-09d45b25-6664-4023-b7b6-c553c95710cf", "question": "Ordering message 20", "plan": ["Preserve related m
  essage order"], "draft": "The question was: Ordering message 20 The response follows this plan: Preserve related message order."}
  [WRITER-one] Offset committed for ordering-keyed-09d45b25-6664-4023-b7b6-c553c95710cf
```

```
}
(venv) (base) sangah@MacBook-Air-SAL hw08_kafka_pipeline % { date '+[%Y-%m-%d %H:%M:%S %Z] ORDERING WRITER 2'; \
WRITER_INSTANCE=two python -u writer.py; } 2>&1 \
| tee -a ../../reports/hw08/RUN_LOG.txt
[2026-10-29 19:43:43 PDT] ORDERING WRITER 2
[WRITER-two] Listening on s4544_tasks
[WRITER-two] Received partition=1 offset=14 sequence=1 correlation_id=ordering-unkeyed-80ed3ef3-568a-4cb0-b8f6-7d6e3a72c5a7
[WRITER-two] Sending to s4544_drafts: {"correlation_id": "ordering-unkeyed-80ed3ef3-568a-4cb0-b8f6-7d6e3a72c5a7", "question": "Ordering message 1", "plan": ["Preserve related
message order"], "draft": "The question was: Ordering message 1 The response follows this plan: Preserve related message order."}
[WRITER-two] Offset committed for ordering-unkeyed-80ed3ef3-568a-4cb0-b8f6-7d6e3a72c5a7
[WRITER-two] Received partition=1 offset=15 sequence=3 correlation_id=ordering-unkeyed-dc2a3384-36a5-4488-b8cf-98942dfa2b47
[WRITER-two] Sending to s4544_drafts: {"correlation_id": "ordering-unkeyed-dc2a3384-36a5-4488-b8cf-98942dfa2b47", "question": "Ordering message 3", "plan": ["Preserve related
message order"], "draft": "The question was: Ordering message 3 The response follows this plan: Preserve related message order."}
[WRITER-two] Offset committed for ordering-unkeyed-dc2a3384-36a5-4488-b8cf-98942dfa2b47
[WRITER-two] Received partition=1 offset=16 sequence=6 correlation_id=ordering-unkeyed-3cc8efca-1f02-4484-a19b-77d4906ad1ed
[WRITER-two] Sending to s4544_drafts: {"correlation_id": "ordering-unkeyed-3cc8efca-1f02-4484-a19b-77d4906ad1ed", "question": "Ordering message 6", "plan": ["Preserve related
message order"], "draft": "The question was: Ordering message 6 The response follows this plan: Preserve related message order."}
[WRITER-two] Offset committed for ordering-unkeyed-3cc8efca-1f02-4484-a19b-77d4906ad1ed
[WRITER-two] Received partition=1 offset=17 sequence=9 correlation_id=ordering-unkeyed-f5261b90-1a46-4f4b-8b56-6a1aa2dec36a
[WRITER-two] Sending to s4544_drafts: {"correlation_id": "ordering-unkeyed-f5261b90-1a46-4f4b-8b56-6a1aa2dec36a", "question": "Ordering message 9", "plan": ["Preserve related
message order"], "draft": "The question was: Ordering message 9 The response follows this plan: Preserve related message order."}
[WRITER-two] Offset committed for ordering-unkeyed-f5261b90-1a46-4f4b-8b56-6a1aa2dec36a
[WRITER-two] Received partition=1 offset=18 sequence=11 correlation_id=ordering-unkeyed-79bb5833-ee22-4466-b6b4-0d41d98cedcb
[WRITER-two] Sending to s4544_drafts: {"correlation_id": "ordering-unkeyed-79bb5833-ee22-4466-b6b4-0d41d98cedcb", "question": "Ordering message 11", "plan": ["Preserve related
 message order"], "draft": "The question was: Ordering message 11 The response follows this plan: Preserve related message order."}
[WRITER-two] Offset committed for ordering-unkeyed-79bb5833-ee22-4466-b6b4-0d41d98cedcb
[WRITER-two] Received partition=1 offset=19 sequence=14 correlation_id=ordering-unkeyed-f6301378-79d6-49be-b603-315f292afb89
[WRITER-two] Sending to s4544_drafts: {"correlation_id": "ordering-unkeyed-f6301378-79d6-49be-b603-315f292afb89", "question": "Ordering message 14", "plan": ["Preserve related
 message order"], "draft": "The question was: Ordering message 14 The response follows this plan: Preserve related message order."}
[WRITER-two] Offset committed for ordering-unkeyed-f6301378-79d6-49be-b603-315f292afb89
[WRITER-two] Received partition=1 offset=20 sequence=15 correlation_id=ordering-unkeyed-387d4fa7-2879-4427-b8cd-05a71f4f8fe0
[WRITER-two] Sending to s4544_drafts: {"correlation_id": "ordering-unkeyed-387d4fa7-2879-4427-b8cd-05a71f4f8fe0", "question": "Ordering message 15", "plan": ["Preserve related
 message order"], "draft": "The question was: Ordering message 15 The response follows this plan: Preserve related message order."}
[WRITER-two] Offset committed for ordering-unkeyed-387d4fa7-2879-4427-b8cd-05a71f4f8fe0
[WRITER-two] Received partition=1 offset=21 sequence=16 correlation_id=ordering-unkeyed-9fed295d-a563-439a-adde-26a3fe0bdf8f
[WRITER-two] Sending to s4544_drafts: {"correlation_id": "ordering-unkeyed-9fed295d-a563-439a-adde-26a3fe0bdf8f", "question": "Ordering message 16", "plan": ["Preserve related
 message order"], "draft": "The question was: Ordering message 16 The response follows this plan: Preserve related message order."}
[WRITER-two] Offset committed for ordering-unkeyed-9fed295d-a563-439a-adde-26a3fe0bdf8f
[WRITER-two] Received partition=1 offset=22 sequence=17 correlation_id=ordering-unkeyed-8feca512-7e0d-4293-a553-4b8fd92a7060
[WRITER-two] Sending to s4544_drafts: {"correlation_id": "ordering-unkeyed-8feca512-7e0d-4293-a553-4b8fd92a7060", "question": "Ordering message 17", "plan": ["Preserve related
 message order"], "draft": "The question was: Ordering message 17 The response follows this plan: Preserve related message order."}
[WRITER-two] Offset committed for ordering-unkeyed-8feca512-7e0d-4293-a553-4b8fd92a7060
[WRITER-two] Received partition=1 offset=23 sequence=18 correlation_id=ordering-unkeyed-fe91b336-ffda-4b92-b91f-47c8b598c83f
[WRITER-two] Sending to s4544_drafts: {"correlation_id": "ordering-unkeyed-fe91b336-ffda-4b92-b91f-47c8b598c83f", "question": "Ordering message 18", "plan": ["Preserve related
 message order"], "draft": "The question was: Ordering message 18 The response follows this plan: Preserve related message order."}
[WRITER-two] Offset committed for ordering-unkeyed-fe91b336-ffda-4b92-b91f-47c8b598c83f
[WRITER-two] Received partition=1 offset=24 sequence=20 correlation_id=ordering-unkeyed-eb5c8d8a-dac6-4eb6-997b-f53a098c711f
[WRITER-two] Sending to s4544_drafts: {"correlation_id": "ordering-unkeyed-eb5c8d8a-dac6-4eb6-997b-f53a098c711f", "question": "Ordering message 20", "plan": ["Preserve related
 message order"], "draft": "The question was: Ordering message 20 The response follows this plan: Preserve related message order."}
[WRITER-two] Offset committed for ordering-unkeyed-eb5c8d8a-dac6-4eb6-997b-f53a098c711f
```

```
(venv) (base) sangah@MacBook-Air-SAL hw08_kafka_pipeline % { date '+[%Y-%m-%d %H:%M:%S %Z] ORDERING UNKEYED'; \
python -u ordering_test.py; } 2>&1 \
| tee -a ../../reports/hw08/RUN_LOG.txt
[2026-10-29 19:44:08 PDT] ORDERING UNKEYED
[ORDERING] Starting unkeyed run with 20 messages
[ORDERING] Sent sequence=1 keyed=False
[ORDERING] Sent sequence=2 keyed=False
[ORDERING] Sent sequence=3 keyed=False
[ORDERING] Sent sequence=4 keyed=False
[ORDERING] Sent sequence=5 keyed=False
[ORDERING] Sent sequence=6 keyed=False
[ORDERING] Sent sequence=7 keyed=False
[ORDERING] Sent sequence=8 keyed=False
[ORDERING] Sent sequence=9 keyed=False
[ORDERING] Sent sequence=10 keyed=False
[ORDERING] Sent sequence=11 keyed=False
[ORDERING] Sent sequence=12 keyed=False
[ORDERING] Sent sequence=13 keyed=False
[ORDERING] Sent sequence=14 keyed=False
[ORDERING] Sent sequence=15 keyed=False
[ORDERING] Sent sequence=16 keyed=False
[ORDERING] Sent sequence=17 keyed=False
[ORDERING] Sent sequence=18 keyed=False
[ORDERING] Sent sequence=19 keyed=False
[ORDERING] Sent sequence=20 keyed=False
(venv) (base) sangah@MacBook-Air-SAL hw08_kafka_pipeline % { date '+[%Y-%m-%d %H:%M:%S %Z] ORDERING KEYED'; \
ORDERING_KEYED=1 python -u ordering_test.py; } 2>&1 \
| tee -a ../../reports/hw08/RUN_LOG.txt
[2026-10-29 19:44:15 PDT] ORDERING KEYED
[ORDERING] Starting keyed run with 20 messages
[ORDERING] Sent sequence=1 keyed=True
[ORDERING] Sent sequence=2 keyed=True
[ORDERING] Sent sequence=3 keyed=True
[ORDERING] Sent sequence=4 keyed=True
[ORDERING] Sent sequence=5 keyed=True
[ORDERING] Sent sequence=6 keyed=True
[ORDERING] Sent sequence=7 keyed=True
[ORDERING] Sent sequence=8 keyed=True
[ORDERING] Sent sequence=9 keyed=True
[ORDERING] Sent sequence=10 keyed=True
[ORDERING] Sent sequence=11 keyed=True
[ORDERING] Sent sequence=12 keyed=True
[ORDERING] Sent sequence=13 keyed=True
[ORDERING] Sent sequence=14 keyed=True
[ORDERING] Sent sequence=15 keyed=True
[ORDERING] Sent sequence=16 keyed=True
[ORDERING] Sent sequence=17 keyed=True
[ORDERING] Sent sequence=18 keyed=True
[ORDERING] Sent sequence=19 keyed=True
[ORDERING] Sent sequence=20 keyed=True
(venv) (base) sangah@MacBook-Air-SAL hw08_kafka_pipeline %
```

In HW2, build_workflow() created an in-process graph, and graph.stream(initial_state) directly passed state from one step to the next. The agents did not need Kafka topics, correlation IDs, consumer groups, or offset management. In HW8, the agents became separate processes and Kafka topics carried the messages between them. This allowed independent scaling and failure isolation, but required additional responsibilities such as preserving message order, handling duplicate delivery, managing offsets, and implementing retries.

Failure scenario	Before fix	After fix
Duplicate delivery	The same message could create more than one draft.	The Writer detected the repeated `correlation_id` and ignored the duplicate.
Consumer killed mid-work	Automatic commits could lose the message after failure.	Manual commits allowed Kafka to redeliver the uncommitted message after restart.
Poison message	A bad message could repeatedly fail and block later messages.	The message was retried three times, moved to `s4544_dead_letter`, and later messages continued.
Two partitions/two consumers	Related messages could be processed by different consumers and partitions.	Using the entity as the Kafka key kept related messages in one partition and preserved their order.

AI USE

(1) What you used an AI assistant for and what you did yourself;
I used an AI assistant to help me understand the demo code to my domain, and used it to assist me in writing the good code formats.
I ran the commands myself, created the Docker containers and Kafka topics, started the separate processes, reviewed the terminal output, and captured the screenshots and run logs. I also checked that the actual topic names, correlation IDs, entity IDs, partitions, retries, and offsets appeared in the results to confirm that the code is running propertly.

(2) One AI-produced output that was wrong/unsuitable, or one thing you independently verified;
The demo created `AIOKafkaProducer` and `AIOKafkaConsumer` objects inside the `KafkaRPC` constructor. That caused FastAPI to fail with an `aiokafka` error saying that the object must be created inside an async function. This demo code was not compatible with the installed environment, so I edited and created a folder to make it available.
I also independently checked that my assignment used the exact `correlation_id`, `reply_to`, `operation`, and `data` request structure.

(3) How you detected the problem or verified the result;
I detected the FastAPI problem by reading the Docker traceback and seeing that the failure occurred while `KafkaRPC()` was being created during module import. After changing the startup code, I rebuilt the FastAPI image and confirmed that the service stayed running and printed the Kafka startup messages. The work was verified using the terminal output.

(4) What you changed and why it works now;
I changed `KafkaRPC` so its Kafka clients are created inside the asynchronous `start()` method. I added the required `s4544` topic names, correlation ID handling, validation, idempotency,

manual offset commits, bounded retries, dead-letter routing, and partition keys, and this allowed the code to work properly.